

Understanding Python's de-facto Memory Model

ANONYMOUS AUTHOR(S)

Since version 0.9.0, the reference implementation of the Python programming language — CPython— uses a Global Interpreter Lock (GIL) to limit concurrency to only one thread executing at any time. Even if unintended, the presence of the GIL provides some guarantees to multi-threaded Python programs, which have been reliably true at least for the past 35 years. Such guarantees are, unfortunately, not well understood, which cause *some* multi-threaded Python programs to work reliably on all versions of Python since 0.9.0 based on *some* behaviors of the GIL. Without understanding exactly the *de-facto* guarantees historically provided by Python's GIL in practice, removing the GIL may cause existing programs to break unexpectedly for unknown reasons. Even if developers are willing to fix their multi-threaded Python programs, there's little guidance explaining what worked before and why it breaks now.

In this paper, we describe that Python's *de-facto* GIL-based behavior matches *Sequentially Consistency* (SC), based on a formal model of a language that captures the GIL discipline enforced by GIL-based Python interpreters. Our model also explains the conditions under which native function calls are atomic, which include most operations on lists, sets, and dictionaries. We validate that our model matches the observed behavior of existing GIL-based implementations using empirical data from litmus testing (memory model) and linearizability checking (builtin data-structures) on CPython, PyPy, and GraalPy. Furthermore, we manually analyze 100 popular Python repositories that use multiple threads, and find evidence that developers rely on SC behavior for shared variables, and on atomic behavior for most list, set, and dictionary operations.

We extend our testing to Python implementations not based on a GIL: CPython's prototype for PEP 703 (which paves the way for removing the GIL from CPython), Jython, and IronPython. We find that only IronPython is not SC, exposing Python programs to the underlying .NET memory model. We also find that lists, sets, and dictionaries allow new behaviors impossible to observe while using a GIL. We reported our findings to the developers, and used the feedback to perform root-cause analysis on representative violations. One of our reports was a real bug that the developers fixed and backported to earlier Python version. All other issues remain open at the time of writing, as developers consider the implications of such behaviors.

ACM Reference Format:

Anonymous Author(s). 2018. Understanding Python's de-facto Memory Model. *J. ACM* 37, 4, Article 111 (August 2018), 25 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Since version 0.9.0 in 1991 [16], the reference implementation of the Python programming language — CPython [12] — uses a *Global Interpreter Lock (GIL)* to limit concurrency to only one thread executing Python bytecodes at any time. Many other popular implementations of Python — PyPy [40] and GraalPy [33, 47] — use a similar mechanism to ensure the same semantics as CPython. The GIL is intended to protect the internal mechanisms of the Python interpreter. The precise behavior is considered an implementation detail that offers no promises to programs relying on it [1]. However, even if unintended, the presence of the GIL does translate to some guarantees visible to multi-threaded Python programs, which have been reliably true at least for the past 35 years. Such guarantees are, unfortunately, not well understood, which causes two main problems. First, *some* multi-threaded Python programs may work reliably on all versions of Python since

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-735X/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

```

1: data = None          3: # Thread 1          6: # Thread 2
2: ready = False       4: data = ...          7: while not ready:
                        5: ready = True          8:     continue
                        9: assert(data is not None)

```

Fig. 1. Small Python sample involving 2 threads that never results in the assertion failing due to Python's Global Interpreter Lock. Other languages, such as C/C++, and Java, can see the assertion failing.

0.9.0 based on *some* behaviors of the GIL, and others may break on some Python versions. We believe that considering the former programs as incorrect, and needing extra synchronization (e.g., using locks), is not a pragmatic approach as it ignores existing code that has worked reliably for decades. Second, the Python council recently accepted [38] a Python Enhancement Proposal (PEP 703 [15]) that paves the way for removing the GIL from CPython. Without understanding exactly the *de-facto* guarantees historically provided by Python's GIL in practice, removing the GIL may cause the former programs to break unexpectedly for unknown reasons. Developers may then become reluctant to upgrade the version of Python they use. Worse, even if developers are willing to fix their multi-threaded Python programs, there's little guidance explaining what worked before and why it breaks now.

The example in Figure 1 shows a multi-threaded Python program that works reliably because of the GIL, in which Thread 1 computes some data in Line 4 and sets a flag in Line 5, while Thread 2 waits for the data to be ready by checking the flag in Lines 7–8. In most programming languages (e.g., C/C++ [20], Java [27], Go [14]), it is possible that the assertion in Line 9 fails on ARM hardware [6]. However, CPython's internal GIL ensures that the assertion never fails because: (1) Thread 2 can only break out of the loop after Thread 1 sets the flag, (2) Thread 2 can only execute after it acquires the GIL, and (3) Thread 2 can only acquire the GIL after Thread 1 releases it. Events 2 and 3 connect the memory view of threads 1 and 2 following a *release-acquire* [20] or *happens-before* [27] relationship, which in turn ensures that Thread 2 can see all writes that Thread 1 issued by the time it reaches Line 9. Therefore, the assertion in Line 9 never fails in GIL-based Python implementations.

Unlike other languages, Python programs rely heavily on native code. For instance, Python's basic data-structures (lists, sets, and dictionaries) are not implemented in Python and are provided (natively) by the Python interpreter. Furthermore, the main role of many modern Python applications is to orchestrate native code execution via Python's Foreign Function Interface (FFI) (e.g., using numpy and pytorch to perform some machine learning task). Again, the GIL presence has important consequences to the semantics of such native functions. Given that most native functions do not interact with the GIL, they appear to execute atomically. For instance, consider calling `List.sort` on a list of integers. Internally, sorting may use an in-place algorithm that swaps the order of elements as it progresses. However, given that sorting happens while holding the GIL, other threads can either see the initial list or the sorted list, without any intermediate states. Unfortunately, such a guarantee fundamentally breaks when native functions call back into interpreted code. Consider now that we use a custom comparator in `List.sort`, which involves calling Python code for each sorted pair, which in turn may lose the GIL inside the interpreter. Now, some other thread can see an intermediate state of the list being sorted. In fact, we show in Section 4.4 that such a scenario can happen in CPython's PEP 703 prototype, in which other threads can see a list being sorted as empty.

In this paper, we show that Python's *de-facto* GIL-based behavior matches *Sequential Consistency* (SC). In Section 2.1, we define a formal model based on a multi-threaded language with shared variables that relies on the same GIL discipline that GIL-based Python implementations enforce,

which Section 2.2 then proves to be SC. Using well established research on describing memory models via litmus testing [4, 5, 9, 17, 45], we implement a set of litmus tests for Python and execute them on all popular Python implementations. Section 3.1 shows that such testing finds no weak behaviors (*i.e.*, violations of SC) on any GIL-based implementation, which provide empirical evidence that our formal model aligns with the observable behavior of GIL-based interpreters. Furthermore, we manually analyze existing multi-threaded Python code in Section 5, and find that developers indeed expect Python to be SC.

This paper also establishes the conditions for native function atomicity based on the GIL. Section 2.3 extends our formalization to functions, and shows that native functions (that do not interact with the GIL) are atomic as long as they do not call Python functions (that may lose the GIL as the interpreter executes them). Inspired by well established research on linearizability testing [8, 10, 29, 43], we introduce a new automatic tool to find violations of atomicity (Section 4.2) – SNEK.¹ Using SNEK, we conduct extensive testing on all operations of lists, sets, and dictionaries that our model describes as atomic, and find all expected operations to be indeed atomic in Section 4.3 (*i.e.*, no violations of linearizability), which amounts to empirical evidence that our formal model matches the observable behavior of GIL-based implementations. We manually analyzed existing multi-threaded Python code and find that developers expect basic Python data-structures to match our proposed atomic semantics (Section 5).

Given the recent efforts to remove the GIL from CPython in PEP 703 [15], we extend our empirical testing to implementations not based on a GIL: CPython with experimental support for PEP 703, Jython [35], and IronPython [30]. Our results (Section 3.1) find that both PEP 703's prototype and Jython show no weak behaviors and thus are consistent with SC, but IronPython exposes the underlying .NET memory model [19, 42] to the Python programs it executes, which in turn results in observable weak behaviors. Furthermore, SNEK found violations on all non-GIL Python implementations, which we analyze in detail (Section 4.3), report to the respective developers, and describe the root-cause (Section 4.4). As a result, one of the issues we found was fixed and backported to earlier versions of CPython, all other remain open as developers consider the implications of such behaviors.

In summary, this paper makes the following contributions:

- A formal model that proposes that a GIL-based language (such as Python) is Sequentially Consistent, that also explains the conditions for atomicity for native/builtin functions.
- An empirical validation that the proposed model aligns with the observable behaviors of GIL-based Python, using existing Python interpreters CPython, PyPy, and GraalPy, relying on litmus testing Python's memory model and linearizability testing Python's basic data-structures (lists, sets, and dicts).
- A manual analysis of 100 popular repositories that contain multi-threaded Python code, that finds evidence that developers rely on SC-like behavior for shared variables and on atomic behavior for most list, set, and dictionary operations.
- Empirical evidence that Python interpreters without a GIL allow behaviors not possible in GIL-based interpreters. We find that the current CPython prototype for PEP 703 and Jython shows no weak behaviors and thus are consistent with SC, that IronPython breaks SC, and that all interpreters break the atomicity of lists, sets, and dicts observed in GIL-based interpreters. We report on issues opened with the developers, and perform a root-cause analysis on representative issues. As a result of our efforts, one issue was fixed and backported to earlier Python versions, while all others remain open.

¹SNEK: Sequential-consistency and linearizability checker

Upon acceptance, we will submit an artifact to the AEC that can replicate all the experiments (and respective results) described in this document.

2 GIL-based Python de-facto Memory Model

In this section, we formalize the memory model of a GIL-based language (similar to Python's GIL-based interpreters) as Sequential Consistency (SC) based on the classical formalism [2, 22] and inspired by the Java Memory Model [27, 32]. Section 2.1 describes our basic execution model around a single Global Interpreter Lock (GIL), which Section 2.2 shows to be Sequentially Consistent (Theorem 2.15). Section 2.3 then extends the base language with functions and shows the conditions under which functions are atomic (Theorems 2.23 and 2.25) or not (Corollary 2.24).

2.1 Execution Model

We start by defining our execution model in which all threads in well-formed executions read and write memory only while holding the GIL. Furthermore, the GIL provides the expected mutual exclusion between threads. Our model matches the discipline around acquiring/releasing the GIL that is automatically enforced by the Python interpreter, which ensures mutual exclusion between threads and is central to our reasoning about visibility and ordering of memory operations among threads.

Definition 2.1 (Threads, Locations, Values). Programs have a finite set of threads uniquely identified $\mathbb{T}$, a set of all memory locations uniquely identified $\mathbb{L}$, and a set of values $\mathbb{V}$ that each memory location may hold.

Definition 2.2 (Actions). We define the following actions for thread $t \in \mathbb{T}$, value $v \in \mathbb{V}$, and location $l \in \mathbb{L}$: (1) t **reads** v from l , denoted $Rd(t, l, v)$, (2) t **writes** v to l , denoted $Wr(t, l, v)$, (3) t **acquires** the GIL, denoted $AcqGIL(t)$, and (4) t **releases** the GIL, denoted $RelGIL(t)$.

Let $\mathbb{A}$ be the set of all actions. Given an action $a \in \mathbb{A}$, we define $tid(a)$ as:

$$tid(Rd(t, l, v)) = t \quad tid(Wr(t, l, v)) = t \quad tid(AcqGIL(t)) = t \quad tid(RelGIL(t)) = t$$

For each location $l \in \mathbb{L}$, let $Wr(t, l, v_0)$ be a distinguished initializing write with an initial value v_0 by some thread t . Let $Init = \{Wr(t, l, v_0) \mid l \in \mathbb{L}\}$.

Definition 2.3 (Program Order). Program order $\rightarrow_{po} \subseteq \mathbb{A} \times \mathbb{A}$ is a relation over actions. Consider actions $a, b, c \in \mathbb{A}$. Program order is **intra-thread only** ($a, b \in \rightarrow_{po} \Rightarrow tid(a) = tid(b)$), has a **strict partial order** that is irreflexive $a \not\rightarrow_{po} a$ and transitive ($a \rightarrow_{po} b \wedge b \rightarrow_{po} c \Rightarrow a \rightarrow_{po} c$). Program order is also **total per thread**: considering any two different actions $a \neq b$ in the same thread $tid(a) = tid(b)$, then $((a \rightarrow_{po} b) \vee (b \rightarrow_{po} a))$.

Definition 2.4 (Executions). An *execution* is a tuple $E = (A, \rightarrow_{po})$ where $A \subseteq \mathbb{A}$ is a finite set of actions, and $\rightarrow_{po} \subseteq A \times A$ is the program order relation. There is a fresh thread t_{Init} that performs initializing writes such that $Init = \{Wr(t_{Init}, l, v_0) \mid l \in \mathbb{L}\}$, and $Init \subseteq A$. We denote all threads in the execution as $T = \{tid(a) \mid a \in A\}$.

We refer to A_t as all actions by a particular thread $t \in \mathbb{T}$: $A_t = \{a \in A \mid tid(a) = t\}$, and A_M as the set of read and write memory actions by all threads: $\{a \in A \mid a = Rd(t, l, v) \vee a = Wr(t, l, v)\}$.

Definition 2.5 (Matched GIL Region). For an execution E , a pair of actions $(a_a, a_r) \in A_t \times A_t$ is a *matched GIL region* of thread $t \in \mathbb{T}$ iff: (i) they form an acquire/release pair such that $a_a = AcqGIL(t)$ and $a_r = RelGIL(t)$, (ii) acquire takes place before release in program order $a_a \rightarrow_{po} a_r$, and (iii) there is no GIL action strictly between a_a and a_r : $\nexists a \in A_t. (a_a \rightarrow_{po} a \rightarrow_{po} a_r \wedge (a = AcqGIL(t) \vee a = RelGIL(t)))$.

Definition 2.6 (GIL slice). For a matched GIL region (a_a, a_r) of thread t , a $Slice(a_a, a_r)$ is a set containing the actions immediately surrounded by a_a and a_r : $\{a \in A_t \mid (a = a_a \vee a_a \rightarrow_{po} a) \wedge (a = a_r \vee a \rightarrow_{po} a_r)\}$.

Definition 2.7 (GIL-Well-Formed Execution). An execution E is *GIL-well-formed* if for every thread $t \in (T \setminus \{t_{Init}\})$ and every memory action of thread t , reads and writes only happen between acquire and release: $\forall a \in A_t \cap A_M. \exists a_a, a_r \in A_t, a \in A_t \cap A_M. a_a = AcqGIL(t) \wedge a_r = RelGIL(t) \wedge a \in Slice(a_a, a_r)$.

Definition 2.8 (GIL Synchronization Order). We refer to A_{GIL} as all **synchronization actions** by any thread t involving the GIL: $\{a \in A \mid a = AcqGIL(t) \vee a = RelGIL(t)\}$.

A **synchronization order** for the GIL is a strict total order $\rightarrow_{GIL} \subseteq A_{GIL} \times A_{GIL}$ that respects the per-thread order of GIL operations: $\rightarrow_{po} \cap (A_{GIL} \times A_{GIL}) \subseteq \rightarrow_{GIL}$.

Definition 2.9 (GIL mutual exclusion). We say that E satisfies *GIL mutual exclusion* iff for every GIL slice $Slice(a_a, a_r)$ of every thread t , no GIL action of any other thread appears strictly between a_a and a_r in $\rightarrow_{GIL}$:

$$\forall t \in T. \forall a_a, a_r \in A_t. (a_a, a_r) \text{ is matched} \Rightarrow \neg \exists t' \neq t. \exists a' \in (A_{t'} \cap A_{GIL}). (a_a \rightarrow_{GIL} a' \rightarrow_{GIL} a_r)$$

2.2 GIL ensures Sequential Consistency

In this section, we show that the GIL discipline introduced in Section 2.1, which matches how the Python interpreter enforces GIL usage, results in programs that are sequentially consistent (Theorem 2.15).

Definition 2.10 (Sequential consistency [22]). An execution E is *sequentially consistent (SC)* iff there exists a strict total order $\rightarrow_{to} \subseteq A_M \times A_M$ such that:

- (1) **Program order preserved:** $\rightarrow_{po} \cap A_M \times A_M \subseteq \rightarrow_{to}$.
- (2) **Reads see the last write:** for every read $r = Rd(t, l, v) \in A_M$, there exists a write $w \in A_M$ to the same location l such that: (i) $w \rightarrow_{to} r$, (ii) w writes value v to l , and (iii) there is no write w' to l with $w \rightarrow_{to} w' \rightarrow_{to} r$.

Definition 2.11 (Slice order induced by the GIL). Consider any two matched GIL regions (a_a, a_r) and (b_a, b_r) (not necessarily in the same thread), with slices $Slice(a_a, a_r)$ and $Slice(b_a, b_r)$ (Definition 2.6). We define an order between slices: $Slice(a_a, a_r) \rightarrow_{slice} Slice(b_a, b_r)$ iff $a_a \rightarrow_{GIL} b_a$, where $\rightarrow_{GIL}$ is the strict total order on GIL actions (Definition 2.8).

Definition 2.12 (Global SC order). We define $\rightarrow_{to}$ as the strict total order over A_M obtained by: (i) ordering memory actions by their enclosing slice using $\rightarrow_{slice}$, and (ii) ordering memory actions within the same slice by program order $\rightarrow_{po}$.

Formally, for distinct $a_1, a_2 \in A_M$, let $S(a_1)$ be a (unique, see Lemma 2.13) slice that contains a_1 . If $a_1 \in Init$, treat it as being in a distinguished slice S_{Init} ordered before all others. We define $a_1 \rightarrow_{to} a_2$ iff $(S(a_1) \rightarrow_{slice} S(a_2)) \vee (S(a_1) = S(a_2) \wedge a_1 \rightarrow_{po} a_2)$.

LEMMA 2.13 (UNIQUENESS GIL SLICES). Assume an execution E is GIL-well-formed and satisfies GIL mutual exclusion. Given a thread t , for every memory action $a \in A_t \cap A_M$, there exists a unique matched GIL region (a_a, a_r) of thread t such that $a \in Slice(a_a, a_r)$. Therefore $S(a)$ in Def. 2.12 is well-defined.

PROOF. (Existence.) Let $a \in A_t \cap A_M$. By GIL-well-formedness, there exists a matched GIL region (a_a, a_r) of thread t such that a occurs between a_a and a_r in program order, hence $a \in Slice(a_a, a_r)$ by Def. 2.6.

(Uniqueness.) Suppose $a \in \text{Slice}(a_a, a_r)$ and $a \in \text{Slice}(a'_a, a'_r)$ for two matched GIL regions of t . Then in $\rightarrow_{po}$ we have $a_a \rightarrow_{po} a \rightarrow_{po} a_r$ and $a'_a \rightarrow_{po} a \rightarrow_{po} a'_r$, so the two regions overlap on thread t . This contradicts Definition 2.5(iii), since the acquire of one region would be a GIL action strictly between the acquire and release of the other. Hence $(a_a, a_r) = (a'_a, a'_r)$. $\square$

Definition 2.14 (Read semantics for $\rightarrow_{to}$). Let E be an execution and with a strict total order $\rightarrow_{to} \subseteq A_M \times A_M$. For every read $r = Rd(t, l, v)$, let w be the $\rightarrow_{to}$ -maximal write to location l such that $w \rightarrow_{to} r$. Then $w = Wr(t', l, v')$ implies $v = v'$.

Definitions 2.7–2.9 capture the structural GIL discipline enforced by the interpreter. Definition 2.14 adds the semantic assumption that reads observe the last write in the total order induced by that discipline. Together, these assumptions model the de-facto guarantees provided a Python-like GIL-based interpreter.

THEOREM 2.15 (GIL-WELL-FORMED EXECUTIONS ARE SEQUENTIALLY CONSISTENT). *Let E be a GIL-well-formed execution (Definition 2.7) that satisfies GIL mutual exclusion (Definition 2.9). Assuming the read semantics from Definition 2.14, then E is sequentially consistent in the sense of Definition 2.10.*

PROOF. We show a strict total order $\rightarrow_{to}$ satisfying the two SC conditions.

(1) $\rightarrow_{to}$ is a strict total order. Because the GIL synchronization order $\rightarrow_{GIL}$ is a strict total order on GIL actions (Definition 2.8), it induces a strict total order $\rightarrow_{slice}$ on slices by ordering their acquire events. By GIL-well-formedness, every memory action belongs to exactly one slice: if $a \in \text{Init}$ then $a \in S_{\text{Init}}$; otherwise, by Definition 2.7 and Lemma 2.13, a belongs to exactly one GIL slice. Within a slice, program order $\rightarrow_{po}$ is total for that thread (Definition 2.3). Therefore, for any two distinct memory actions, either their slices are ordered by $\rightarrow_{slice}$, or they lie in the same slice and are ordered by $\rightarrow_{po}$, hence $\rightarrow_{to}$ is total and irreflexive.

(2) $\rightarrow_{to}$ extends per-thread program order on memory actions. Take any $a_1 \rightarrow_{po} a_2$ with $a_1, a_2 \in AM$. By GIL-well-formedness, both actions lie inside some matched GIL regions of the same thread. If they lie in the same slice, then $a_1 \rightarrow_{to} a_2$. If they lie in different slices of the same thread, then their enclosing acquires are ordered in $\rightarrow_{GIL}$ (because $\rightarrow_{GIL}$ respects per-thread order of GIL actions), hence their slices are ordered by $\rightarrow_{slice}$, so again $a_1 \rightarrow_{to} a_2$.

(3) Reads observe the last preceding write. Let $r = Rd(t, l, v)$ be any read, and $s = S(r)$ be the (unique) GIL slice that contains r (Definition 2.12; well-defined by Lemma 2.13). Let w be the $\rightarrow_{to}$ -maximal write to location l , as per Definition 2.14. By construction of $\rightarrow_{to}$, all actions in slices strictly earlier than s precede all actions in s , and within s the order $\rightarrow_{to}$ agrees with program order $\rightarrow_{po}$. Therefore, w is exactly the last write to l that occurs before r in the serialized order. By Definition 2.14, r reads the value written by w , so $w = Wr(_, l, v)$. This is exactly Definition 2.10(2). Therefore, the SC read condition holds for r .

Having established (1), (2), and (3), E is sequentially consistent. $\square$

2.3 GIL conditions for native function atomicity

This section starts by expanding the execution model we introduce in Section 2.1 with functions that can be called, returned from, and nested in well-formed executions. We align our model with how Python handles native functions, either built into the interpreter or called via Foreign Function Interface (FFI), based on the observation that most native functions do not interact with the GIL. Definition 2.19 considers such native functions as *NoGIL*, introduces *leaf* functions that do not call back into the Python interpreter (e.g., `List.reverse`), and defines *blocking functions* that voluntarily release the GIL but reacquire it before performing any Python-visible write (e.g.,

Socket.accept while waiting for a connection). Interestingly, non-leaf NoGIL functions can call the Python interpreter from native code (e.g., List.sort with a custom comparator written in Python). We then show that leaf NoGIL functions are atomic (Theorem 2.23), non-leaf NoGIL functions are not atomic (Corollary 2.24), and blocking functions are write-atomic (Theorem 2.25).

Definition 2.16 (Call actions). Let $\mathbb{F}$ be a (finite) set of function names. We extend the set of actions with $CALL(t, f)$ and $RETURN(t, f)$ for $t \in \mathbb{T}$ and $f \in \mathbb{F}$. These actions participate in program order ($\rightarrow_{po}$) just like the actions defined in Section 2. For convenience, we extend $tid(\cdot)$ so that $tid(CALL(t, f)) = t$ and $tid(RETURN(t, f)) = t$.

Let $A_{funmem} = (A \setminus A_{GIL}) \cup Init$, and a fixed strict total order $\rightarrow_{ser} \subseteq A_{funmem} \times A_{funmem}$ that (only) extends the SC order $\rightarrow_{to}$ to $CALL/RETURN$ actions:

- (1) $\rightarrow_{po} \cap (A_{funmem} \times A_{funmem}) \subseteq \rightarrow_{ser}$
- (2) $\forall a, b \in A_M, a \rightarrow_{ser} b \iff a \rightarrow_{to} b$.

Definition 2.17 (Call matching and Function bodies). A call matching for a thread $t \in \mathbb{T}$ and some function $f \in \mathbb{F}$ is a partial function: $match_t : \{a \in A \mid a = CALL(t, f) \text{ for some } f\} \rightarrow \{a \in A \mid a = RETURN(t, f) \text{ for some } f\}$ such that, for every call action $a_c = CALL(t, f)$ in its domain, $match_t$: (i) **preserves the name**, $match_t(a_c) = RETURN(t, f)$ for the same f , (ii) **is forward**, $a_c \rightarrow_{po} match_t(a_c)$, and (iii) **is injective**, $match_t(a_c) = match_t(a'_c) \Rightarrow a_c = a'_c$.

Let $a_c = CALL(t, f)$ be a function invocation, and $a_r = match_t(a_c)$ the respective return action. We define its *function invocation body* as: $Body(a_c) = \{a_c\} \cup \{a \in A_t \mid a_c \rightarrow_{po} a \rightarrow_{po} a_r\} \cup \{a_r\}$.

Definition 2.18 (Call-well-formedness). An execution E is *call-well-formed* iff for every thread $t \in \mathbb{T}$ there exists a matching function $match_t$ from call actions of t to return actions of t . For every call action a_c of thread t , let $a_r = match_t(a_c)$, such that:

- (1) $a_c \rightarrow_{po} a_r$, and for any two distinct call actions a_c^1, a_c^2 of t , the corresponding invocation bodies are properly nested: $\neg(a_c^1 \rightarrow_{po} a_c^2 \rightarrow_{po} a_r^1 \rightarrow_{po} a_r^2)$. Moreover, $match_t$ is a bijection between call actions and return actions of t : every call has exactly one matching return, and every return has exactly one matching call.
- (2) For every action a of thread t that is not a call or return, there exists an innermost invocation a'_c whose body contains a : $a \in Body(a_c) \wedge \forall a''_c. (a \in Body(a''_c) \Rightarrow Body(a'_c) \subseteq Body(a''_c))$.
- (3) a_c is the first action of exactly one invocation body $Body(a_c)$, and a_r is the last action of exactly one invocation body. There exists a unique a_c such that $match_t(a_c) = a_r$.

Definition 2.19 (Leaf, NoGIL, and blocking functions). Given a call-well-formed E , a function $f \in \mathbb{F}$. Considering every invocation $a_c = CALL(t, f)$, we define function f as:

- (1) *Leaf* if it does not call other functions: $\forall a \in Body(a_c). \nexists g \in \mathbb{F}. a = CALL(t, g) \wedge a \neq a_c$
- (2) *(Shallow) NoGIL* if it contains no GIL actions in its own dynamic extent (i.e., it does not contain GIL actions, but functions it calls may contain GIL actions). Let $LocalBody(a_c) \subseteq Body(a_c)$ be the set of actions in $Body(a_c)$ that are not contained in the body of any nested call: $\{a \in Body(a_c) \mid \nexists a'_c \in Body(a_c). a'_c = CALL(t, g) \wedge a'_c \neq a_c \wedge a \in Body(a'_c)\}$. Then f is NoGIL iff: $\forall a \in LocalBody(a_c). a \neq AcqGIL(t) \wedge a \neq RelGIL(t)$.
- (3) *Blocking* if all writes are after the last $AcqGIL$:

$$\forall a \in Body(a_c). (a = AcqGIL(t)) \Rightarrow (\nexists w \in Body(a_c). w = Wr(t, l, v) \wedge w \rightarrow_{po} a).$$

Definition 2.20 (Visibility of writes). Let E be an execution and with a strict total order $\rightarrow_{ser} \subseteq A_{funmem} \times A_{funmem}$. Given an action $a \in A_{funmem}$ and a write $w \in A_M$ with $w = Wr(t', l, v)$, we define the **visibility predicate** $Visible(a, w) = (w \rightarrow_{ser} a)$ meaning that: at action a , the effect of write w is visible to thread $t = tid(a)$.

Definition 2.21 (Write atomicity). Let E be a call-well-formed execution, and $a_c = \text{CALL}(t, f)$ be an invocation. We define the *write-set* $Wset(a_c)$ as $\{w \in \text{Body}(a_c) \mid w = Wr(t, l, v) \text{ for some } l, v\}$.

We say this invocation is *write atomic* if, for every action $a \in A_M$ executed by a different thread ($\text{tid}(a) \neq t$), all writes of the invocation have identical visibility at a :

$$\forall a \in A_M. \text{tid}(a) \neq t \Rightarrow \forall w_1, w_2 \in Wset(a_c). \left(\text{Visible}(a, w_1) = \text{Visible}(a, w_2) \right)$$

Definition 2.22 (Read atomicity). Let E be a call-well-formed execution, and $a_c = \text{CALL}(t, f)$ be an invocation. We define the *read-set* $Rset(a_c)$ as $\{r \in \text{Body}(a_c) \mid r = Rd(t, l, v) \text{ for some } l, v\}$.

We say this invocation is *read atomic* if every read action $r \in Rset(a_c)$ only has visibility of: (i) writes that occurred before the call, or (ii) writes performed by the invocation itself:

$$\forall r \in Rset(a_c). \forall w \in A_M. \left(\text{Visible}(r, w) \Rightarrow (\text{Visible}(a_c, w) \vee (w \in Wset(a_c) \wedge w \rightarrow_{po} r)) \right).$$

THEOREM 2.23 (LEAF NOGIL FUNCTIONS ARE READ- AND WRITE-ATOMIC UNDER SC). Let E be an execution that is: call-well-formed, GIL-well-formed, observes GIL mutual exclusion, and is sequentially consistent. Let $f \in \mathbb{F}$ be a function that is both leaf and NoGIL, and let $a_c = \text{CALL}(t, f)$ be any invocation of f with matching return $a_r = \text{match}_t(a_c)$. Then this invocation is both write-atomic and read-atomic.

PROOF SKETCH. Because the execution is GIL-well-formed, every memory action occurs while the thread holds the GIL. Given that f is leaf and NoGIL, there is no *AcqGIL*, *RelGIL*, or nested *CALL* actions in $\text{Body}(a_c)$; only *Rd*, *Wr*, the initial *CALL*, and the final *RETURN*. Furthermore, thread t does not release the GIL at any point between a_c and a_r . By mutual exclusion of the GIL, no other thread can perform memory actions during this interval. Under sequential consistency, this implies that the global order cannot interleave any other thread's memory actions between memory actions of $\text{Body}(a_c)$.

Write atomicity. Let a be any action of another thread ($\text{tid}(a) \neq t$). Given that no other thread can execute memory actions while t holds the GIL in $\text{Body}(a_c)$, a must occur either entirely before or entirely after all writes in $Wset(a_c)$ in the SC order. Hence either all writes in $Wset(a_c)$ are visible at a , or none are, so all writes have identical visibility at a (Definition 2.21).

Read atomicity. Consider any read $r \in Rset(a_c)$. Given that no other thread executes memory actions between a_c and r , the only writes that can be visible at r are: (i) writes that were already visible at a_c , or (ii) writes performed earlier in this invocation, i.e., $w \in Wset(a_c)$ with $w \rightarrow_{po} r$. This is exactly Definition 2.22. $\square$

COROLLARY 2.24 (NON-LEAF NOGIL FUNCTIONS ARE NEITHER READ- NOR WRITE-ATOMIC). Let E be an execution that is: call-well-formed, GIL-well-formed, observes GIL mutual exclusion, and is sequentially consistent. There exists a function f such that: (i) f is non-leaf and NoGIL, and (ii) an invocation $a_c = \text{CALL}(t_1, f)$ is neither write-atomic nor read-atomic.

COUNTEREXAMPLE. Let f be non-leaf, so its invocation $a_c = \text{CALL}(t_1, f)$ contains a nested call $a'_c = \text{CALL}(t_1, g)$ to some function g . We can construct an execution E as follows.

Thread t_1 calls f and performs a write w_1 to location x , then calls g , whose body contains a *RelGIL*(t_1) followed later by an *AcqGIL*(t_1). During such an interval, another thread t_2 runs while holding the GIL and performs a write w_2 to the same location x . After g returns, t_1 (still in f) performs a second write w_3 to x and then returns.

Given that f is NoGIL, the GIL actions in g do not violate Definition 2.19 for f . Under SC, there is a total order $w_1 <_{sc} w_2 <_{sc} w_3$. Consider the action $a = w_2$. At a , the write w_1 is visible but w_3 is not, so two writes from $Wset(a_c)$ have different visibility at the same external action a , contradicting write-atomicity (Definition 2.21).

For read-atomicity, additionally let t_1 perform a read r of x immediately after returning from g but before executing w_3 , and let r read the value written by w_2 . Then w_2 is visible at r but $w_2 \notin WSet(a_c)$ and w_2 was not visible at a_c , so r observes a write that is neither visible at invocation nor written earlier in the invocation, contradicting read-atomicity (Def. 2.22). $\square$

THEOREM 2.25 (LEAF BLOCKING FUNCTIONS ARE WRITE-ATOMIC). *Let E be an execution that is: call-well-formed, GIL-well-formed, observes GIL mutual exclusion, and is sequentially consistent. Let f be a function that is both leaf and blocking (Definition 2.19). Then every invocation $a_c = CALL(t, f)$ of f is write-atomic (Definition 2.21).*

PROOF SKETCH. Consider an invocation $a_c = CALL(t, f)$ with matching return a_r . If $WSet(a_c) = \emptyset$, write-atomicity holds trivially, so assume $WSet(a_c) \neq \emptyset$.

Let $a_{last} \in Body(a_c)$ be the last $AcqGIL(t)$ action in the invocation (if no acquire occurs in $Body(a_c)$, take $a_{last} = a_c$; by GIL-well-formedness the thread already holds the GIL when executing memory actions in $Body(a_c)$). Since f is *blocking*, every write in $WSet(a_c)$ occurs after a_{last} in program order. Because f is *leaf*, there are no nested calls inside $Body(a_c)$, hence there is no callee that could release the GIL on t 's behalf; thus from a_{last} until the last write of the invocation, thread t performs the entire write region while holding the GIL.

By mutual exclusion of the GIL, no other thread can perform memory actions during the interval in which t holds the GIL and executes the writes of $WSet(a_c)$. Therefore, in the SC total order, the writes in $WSet(a_c)$ form a contiguous block with no interleaving memory actions from other threads.

Now consider any action a with $tid(a) \neq t$. Either: (i) a occurs before this contiguous block of writes in the SC order, in which case none of the writes in $WSet(a_c)$ are visible at a , or (ii) a occurs after the block, in which case all writes in $WSet(a_c)$ are visible at a . Thus all writes in $WSet(a_c)$ have identical visibility at every external action a , which is exactly write-atomicity (Definition 2.21). $\square$

3 Validating Python's Sequential Consistency

In this section, we validate whether the formalization of Python's memory model we propose in Section 2.2 matches the set of observable behavior produced by existing Python implementations. We used the most recent versions of the Python interpreters at the time of writing: CPython 3.14 with and without experimental support for removing the GIL, PyPy version 7.3.20, GraalPy version 25.0.2, Jython version 2.7.4, and IronPython version 3.4.2. One option is to build on our formal model and attempt to verify each Python implementation, which requires verification tools that support C/C++, Java, and C#/.NET. Verifying in such way even a single implementation represents an enormous research challenge that may require improving the state-of-the-art on existing verification tool-chains. The goal of this document is to check many such implementations, so formal methods do not scale.

Instead, we validate that existing implementations match our formal model using empirical evidence obtained via *litmus testing*, which is a well-established approach to find violations of Sequential Consistency (i.e., to find weak-behaviors) [4, 5, 9, 17, 45]. Even though testing cannot prove the absence of bugs, we believe that litmus testing can validate whether the underlying implementation attempts to provide SC or not. We interpret failed tests as observable evidence that the implementation provides a different memory model that does not match SC. Given the extensiveness of our testing across so many different implementations, we consider the absence of failed tests as empirical evidence consistent with our proposed SC memory model.

Throughout the rest of this document, we describe experiments taking place in two possible machines. The **x86_64** machine is equipped with 4 sockets, each holding an Intel(R) Xeon(R) Gold

#Vars	#Thread 1	#Thread 2	#Vars	#Thread 1	#Thread 2
a=0	a=1	b=1	a=0	a=1	b=2
b=0	tmp1=b	tmp2=a	b=0	b=1	tmp=a
weak (tmp1 == 0 and tmp2 == 0)			weak (a == 0 and b == 2)		
(a) Store-buffer litmus test			(b) Read litmus test		

Fig. 2. Two litmus tests. Each test shares the variables listed, initialized as shown. Each test executes both threads concurrently. Once both threads complete, predicate **weak** denotes violations of SC (*i.e.*, the test observed a weak behavior).

5318H CPU (18 physical cores) and 194GB of RAM. Even though NUMA architectures present an interesting environment for memory consistency testing, we consider NUMA out of scope for this work. As such, we bind our experiments on the x86_64 machine to a single node. We also used an ARM machine NVIDIA Jetson Orin Nano Super Developer Kit, equipped with 6 CPUs and 8GB of RAM. Both machines run Ubuntu 22.04.3 LTS.

3.1 Litmus testing

An established method to test memory models is via *litmus testing* [4, 5, 9, 17, 45], which uses small tests to observe weak behaviors — behaviors that cannot be explained by Sequential Consistency (SC). Figure 1 shows one such test called *Message Passing (MP)*, and SC guarantees that the assertion in Line 9 never fails. Figure 2 shows two more litmus tests — *Store-buffer (SB)* and *Read (R)* — which should never terminate with the predicate **weak** observing true. We call executions that observe values not explained by SC *weak behaviors*.

Weak behaviors are intrinsically related with the underlying hardware, as some architectures may ensure memory orderings that prevent weak behaviors. For instance, failing the assertion in Line 9 can be observed in ARM [6] but not in x86/x86_64 [41]. On the other hand, both threads in Figure 2 can observed weak behaviors in both architectures. To test Python’s memory model we use 5 total litmus tests that can cover all memory and cache effects in ARM and x86 using only 2 threads [5]: Read (R), Store (S), Store Buffering (SB), Message Passing (MP), 2+2 write (2+2W). We also used Load Buffering (LB), but we could not observe any weak effect with it, so we ignore it from our discussion.

We first implemented all litmus tests in C++, to ensure the sanity of our testing hardware by observing that the expected weak behaviors can indeed be triggered. To improve the chance of observing weak behaviors, and bring it in line with state-of-the-art tool litmus [5], we added barriers to align the execution of threads at the start of each litmus test (*e.g.*, before lines 4 and 7 in Figure 1 and before the first write on each thread in Figure 2). Table 1 shows the weak behaviors we observed in each architecture.

The main goal of the C++ litmus tests is simplicity, so that we can port them directly to Python with little modifications. We did so, and we considered possible ways in which two variables can be shared by many threads: *global variables*, and *nonlocal variables*. Global variables are straightforward and available in most programming languages as variables that exist for the lifetime of the program and can be accessed anywhere directly by name. We implemented the 5 tests describe above for both global and nonlocal variables, resulting in 10 total tests. We took special care to pad the variables in the tests to eliminate false-sharing [7]. Table 2 shows the weak behaviors we observed for each Python implementation in each architecture.

Table 1. Weak behaviors observed per litmus test using our straightforward C++ implementation. Each row reports the percentage observed in 1M test executions, averaged over 10 runs, with enough independent thread pairs to saturate all cores in each testing machine.

Architecture	Version	2+2W	MP	R	S	SB
ARM	C++	0.06572%	0.25231%	8.12296%	0.00012%	16.15401%
x86_64	C++	0	0	0.02346%	0	0.03636%

Table 2. Weak behaviors observed per litmus test on six Python interpreters, using global (G) and non-local (NL) variables. Each row reports the total observed in 3M test executions, averaged over 10 runs, with enough independent thread pairs to saturate all cores in each testing machine. Non-local variables were introduced in Python 3, and Jython only supports Python 2, hence the lack of NL results. PEP 703 uses the respective CPython prototype.

Architecture	Interpreter		2+2W	MP	R	S	SB
ARM	CPython	G	0	0	0	0	0
		NL	0	0	0	0	0
	PyPy	G	0	0	0	0	0
		NL	0	0	0	0	0
	GraalPy	G	0	0	0	0	0
		NL	0	0	0	0	0
x86_64	CPython	G	0	0	0	0	0
		NL	0	0	0	0	0
	PyPy	G	0	0	0	0	0
		NL	0	0	0	0	0
	GraalPy	G	0	0	0	0	0
		NL	0	0	0	0	0

(a) GIL-based Python interpreters

Architecture	Interpreter		2+2W	MP	R	S	SB
ARM	PEP 703	G	0	0	0	0	0
		NL	0	0	0	0	0
	Jython	G	0	0	0	0	0
		NL	—	—	—	—	—
	IronPython	G	0	1.1	8.1	0	34.4
		NL	0	0	0	0	0
x86_64	PEP 703	G	0	0	0	0	0
		NL	0	0	0	0	0
	Jython	G	0	0	0	0	0
		NL	—	—	—	—	—
	IronPython	G	0	0	0.7	0	41.0
		NL	0	0	0.1	0	1.1

(b) Python interpreters without a GIL

Sequential Consistency for GIL-based Python interpreters. The results in Table 2a show that we found **no weak behaviors for GIL-based Python implementations**.

Sequential Consistency for Python interpreters without a GIL.. The memory model we propose in Section 2 targets GIL-based Python implementations, which include the reference implementation CPython. Even though not explained by our memory model, we reran our litmus testing protocol on Python implementations that do not rely on a GIL to check if such implementations try to remain compatible, or provide a different memory model that allows for weak behaviors. Table 2b shows the results. We can see that **all Python implementations without a GIL, except IronPython, showed no weak behaviors in our litmus testing**.

3.2 Discussion

Our testing methodology followed best practices of litmus testing for memory models [5]. Even though testing cannot prove the absence of weak behaviors, the results we report, obtained after extensive testing, is a strong indication that the underlying GIL-based interpreters showed no weak behaviors in our litmus testing, and thus match the formal model we propose in Section 2.2.

Furthermore, most implementations not based on a GIL (including the current CPython prototype for PEP 703) also remain SC. Jython's results make sense, as its documentation [21] describes the memory model for Python as Sequential Consistency (SC), which we echo in Section 2. Jython's documentation provides a simple argument based on how both Jython and CPython store global variables on a globally accessible dictionary with atomic semantics, which translates to SC as reading/writing from global variables involves manipulating an atomic data-structure. Our argument in Section 2 is more generalizable as it does not rely on storage constraints for how variables are

implemented. For instance, GraalPy compiles Python directly to Java using its polyglot support [47] without such an atomic dictionary of global variables.

The text of PEP 703 [15] and the documentation of IronPython [11] do not describe any memory model. These documents do not contain any examples of multiple threads reading/writing shared variables and the respective allowed/disallowed values, or any other discussion of how one thread can observe memory operations concurrently made by another thread. Yet, the current prototype of PEP 703 remains SC. We believe removing SC will result in breaking existing Python multi-threaded programs that rely on it (even if implicitly). Section 5 shows that such programs do exist and are somewhat common.

IronPython is the only implementation tested that breaks SC. Comparing the results in Tables 1 and 2b suggests that IronPython exposes the underlying .NET memory model [19, 42] to Python programs. It is unclear if IronPython’s developers intended this behavior or if it is an unintended implementation detail, we believe it is the latter. We reported this issue to the IronPython developers as a bug report, which remains open and unanswered at the time of writing. In fact, other IronPython developers complain about surprising behavior when executing multiple threads, which changes from one version of IronPython to the another.²

Performance Cost of Sequential Consistency. We can approximate the potential cost of enforcing sequential consistency by drawing a parallel with literature on enforcing a *volatile by default* implementation of Java [24, 25], which reports performance overheads in the order of 28% for x86 and 37%–51% for ARM. Of course, Java is a highly-performant JIT-compiled language with decades of effort towards its multi-tiered compiler optimizations, so the cited results do not directly transfer to Python. However, such results indicate that stronger memory semantics are achievable in practice. In Python, the cost of SC must be weighed against backwards compatibility: our results suggest that existing multithreaded Python programs may already rely on SC, even if implicitly, and removing it risks breaking code that has worked reliably for decades. We therefore argue that paying some performance cost to preserve SC is worthwhile.

4 Validating Python’s atomicity for built-in functions

Unlike other managed languages such as Java, Python’s basic “built-in” data-structures are implemented natively as part of the interpreter — specifically lists, sets, and dictionaries. Section 2.3 introduces *NoGIL* functions (Definition 2.19b) as functions that do not interact with the GIL in any way. We note that our definition of NoGIL is shallow: the function itself does not interact directly with the GIL, but may still call functions defined in Python, which themselves do interact with the GIL. We consider that all native/built-in functions are NoGIL for the remainder of this section, unless we say otherwise. This is a reasonable assumption, as the whole implementation of lists, sets, and dictionaries does so. Section 2.3 also introduces *leaf* functions (Definition 2.19a) as functions that do not call other (Python) functions. If a native function calls another native function, we still consider that a leaf function if neither calls back into Python code.

The main consequences of our formalization of Python native functions in a GIL-based implementation is in Theorem 2.23 and Corollary 2.24. Theorem 2.23 states that leaf NoGIL functions are atomic to the rest of the Python code in the same program, because they execute to completion while holding the GIL and without any opportunity to release it. Corollary 2.24 states that the ability for a native function to call a Python function results in losing atomicity, as the called Python function may release the GIL and let the rest of the Python program observe an intermediate result of the native function being called. We note that Python code checks-in with the interpreter

²<https://github.com/IronLanguages/ironpython3/issues/1826>

Table 3. Non-GIL operations common to two or more basic Python data-structures. Superscript denote “dunder” methods that may be called on the elements.

Dict	List	Set
<code>__contains__^{eh}</code>	<code>__contains__^e</code>	<code>__contains__^{eh}</code>
<code>__delitem__^{deh}</code>	<code>__delitem__</code>	—
<code>__eq__^{eh}</code>	<code>__eq__^e</code>	<code>__eq__^{eh}</code>
<code>__getitem__^{eh}</code>	<code>__getitem__</code>	—
<code>__ior__^{eh}</code>	—	<code>__ior__^{eh}</code>
<code>__len__</code>	<code>__len__</code>	<code>__len__</code>
<code>__ne__^{eh}</code>	<code>__ne__^e</code>	<code>__ne__^{eh}</code>
<code>__new__</code>	—	<code>__new__</code>
<code>__or__^{eh}</code>	—	<code>__or__^{eh}</code>
<code>__reversed__</code>	<code>__reversed__</code>	—
<code>__sizeof__</code>	<code>__sizeof__</code>	<code>__sizeof__</code>
<code>__str__</code>	<code>__str__</code>	<code>__str__</code>
<code>clear^d</code>	<code>clear^d</code>	<code>clear^d</code>
<code>copy</code>	<code>copy</code>	<code>copy</code>
—	<code>remove^e</code>	<code>remove^{eh}</code>
<code>pop^{eh}</code>	<code>pop^{eh}</code>	<code>pop</code>
<code>update^{deh}</code>	—	<code>update^{deh}</code>

Table 4. Non-GIL operations unique to each basic Python data-structures. `List.sort` takes no arguments, and `List.extend` uses an existing list not mutated by other threads. Superscript denote “dunder” methods that may be called on the elements.

Dict	List	Set
<code>__dir__</code>	<code>__add__</code>	<code>__and__^{eh}</code> <code>add^{eh}</code>
<code>__repr__</code>	<code>__iadd__</code>	<code>__ge__^{eh}</code> <code>difference^{eh}</code>
<code>__ror__^{eh}</code>	<code>__imul__</code>	<code>__gt__^{eh}</code> <code>difference_update^{deh}</code>
<code>__setattr__</code>	<code>__mul__</code>	<code>__iand__^{deh}</code> <code>discard^{deh}</code>
<code>__setitem__^{eh}</code>	<code>__rmul__</code>	<code>__isub__^{deh}</code> <code>intersection^{eh}</code>
<code>fromkeys</code>	<code>__setitem__^{count^d}</code>	<code>__ixor__^{deh}</code> <code>intersection_update^{deh}</code>
<code>get^{eh}</code>	<code>append</code>	<code>__lt__^{eh}</code> <code>isdisjoint^{eh}</code>
<code>items</code>	<code>count^e</code>	<code>__sub__^{eh}</code> <code>issubset^{eh}</code>
<code>keys</code>	<code>extend</code>	<code>__xor__^{eh}</code> <code>issuperset^{eh}</code>
<code>popitem^{deh}</code>	<code>index^e</code>	<code>symmetric_difference^{eh}</code>
<code>setdefault</code>	<code>insert</code>	<code>symmetric_difference_update^{deh}</code>
<code>values</code>	<code>reverse</code>	<code>union^{eh}</code>
	<code>sort^l</code>	

often (e.g., in CPython after executing each bytecode) to potentially release the GIL, so any Python function may release the GIL at any point in its execution.

4.1 Leaf NoGIL methods

To validate that our formal model matches how GIL-based Python works, we consider the implementation of leaf NoGIL functions for lists, sets, and dictionaries that hold integers and/or strings. We focus on the basic data-structures and basic data-types, as virtually all Python programs use them, Tables 3 and 4 list all such functions (methods).

The list of methods we consider includes a number of *double underscore methods* (“dunder”), such as `Set.__contains__`, which the Python interpreter calls to check if element *a* is in set *s*: `a in s`. By default, such dunder methods are also implemented in native code. However, Python developers can redefine them with custom logic in Python, which may cause otherwise leaf methods to become non-leaf and, by Corollary 2.24, lose any atomicity properties. Tables 3 and 4 superscripts denote the first letter of each dunder method that may be called on the elements:

- d: `__del__` may be called anytime that the last reference to an element is lost.
- e: `__eq__` used for establishing equality between elements
- h: `__hash__` obtaining the hash code of an element
- l: `__lt__` comparing the order of elements

4.2 Validating atomicity via linearizability checking

Section 2.3 introduces read- and write-atomicity for functions (Definitions 2.22 and 2.21, respectively), and Theorem 2.23 states that leaf NoGIL functions are both. Such definitions match the semantics of *linearizability* [18], which states that operations over a linearizable data-structure appear to take place atomically at one indivisible point in their execution, sometime between call and return. Before that point, the operation did not take place and no thread can observe it, after that point the operation completed and all threads can observe the full results. At no point can any thread observe partial results of an operation still in-progress. Linearizability allows for modular composability of concurrent methods, as the state of a linearizable data-structure can always be


```

638 10: initial_state = { 1 , 2 , 3 , 4 , 5 , 6 }
639 11:
640 12: ops = [ snek_op(set.add , 1),
641           snek_op(set.intersection_update , {1,5}),
642           snek_op(set.symmetric_difference_update, {1,5,7,8,9}),
643           snek_op(set.__str__ , None, sorted=True),
644           snek_op(set.__ior__ , {2,6}, noret=True) ]
645 17:
646 18: snek_run(initial_state, ops)

```

Fig. 3. Usage of SNEK. Developers provide an initial data structure (Line 10) and an array of operations (Lines 12–16) to `snek_run` (Line 18).

explained by the sequential application of all the operations that took place over that data-structure, totally ordered by their linearization order.

To validate whether existing GIL-based Python implementations of lists, sets, and dicts match our proposed model, we use linearizability checking, building on the large and well established body of work on testing whether data-structures are, in fact, linearizable[8, 10, 29, 43]. We introduce a new random testing framework — SNEK — that automatically finds violations of linearizability for *any* data structure in Python (built-in or not) and automatically generates sample minimal programs that show the violation. Figure 3 shows an example of how developers can use SNEK on built-in sets. First, SNEK needs an initial data structure: Line 10 specifies a set containing all integers in the range 1–6. Second, SNEK requires a list of valid operations on that data structure, which developers can specify using `snek_op`. The first argument is an operation on the data structure, and the second argument is passed as an argument to the operation specified. Lines 12–16 define 5 such operations. Finally, Line 18 starts SNEK, which looks for violations until manually stopped. When SNEK finds a violation, it generates a unique identifier which can be used later to replicate it, minimize it, and generate a Python test that shows it (as shown in Figure 4).

4.2.1 Searching for violations. SNEK’s random testing algorithm works as described below. When it finds a violation, SNEK automatically generates a test as shown in Figure 4, which we use to explain how SNEK works.

Step 1: Choose N operations on that data structure, at random from all the operations provided. In this case, SNEK chooses 2 such operations: `intersection_update` (Line 21) and `symmetric_difference_update` (Line 24).

Step 2: Compute all the sequential permutations of the chosen operations, the value each returns, and the resulting state of the data structure. Line 27 shows the resulting combinations for the operations chosen in our example, identified by their line numbers: 21, 24 results in the set {7, 8, 9}; and 24, 21 results in the empty set — `set()`. Also note that all operations return `None`. Omitted from the example is a `try` block catching all exceptions around each operation (around Line 21 and 24), which SNEK considers as the result when an operation terminates with an exception.

Step 3: Launch one thread per operation. SNEK generates threads that align their execution using a barrier (Lines 20 and 23), then perform one of the chosen operations on the shared data structure, and register the observed return value (Lines 21 and 24). SNEK launches such threads (Line 31) and waits for their termination (Line 32).

Step 4: Check for violations. After all threads finish, SNEK uses object equality to check if the data structure matches any expected state from Step 2, together with all the values observed by all

```

687 19: def t1(b1,s,r):
688 20:     b1.wait()
689 21:     r[0]=s.intersection_update({1,5})
690 22: def t2(b1,s,r):
691 23:     b1.wait()
692 24:     r[1]=s.symmetric_difference_update({1,5,7,8,9})
693 25: def t1t2(i):
694 26:     s = {1,2,3,4,5,6}
695 27:     expected = [ ({7,8,9} , [None,None]), (set() , [None,None]) ]
696 28:     barrier = Barrier(2)
697 29:     res = [0,0]
698 30:     threads = [ Thread(t1(barrier,s,res)), Thread(t2(barrier,s,res)) ]
699 31:     for t in threads: t.start()
700 32:     for t in threads: t.join()
701 33:     assert (s,res) in expected
702 34:
703 35: while True:
704 36:     threads = []
705 37:     for i in range(0,WORKERS):
706 38:         threads.append(Thread(t1t2(n)))
707 39:     for t in threads: t.start()
708 40:     for t in threads: t.join()
709
710
711

```

Fig. 4. Sample test automatically generated by SNEK for a violation of linearizability. SNEK found this example originally with 4 operations, and minimized to the 2 operations shown in Lines 34 and 37.

threads (Line 33). SNEK reports any unexpected results as violations. In our example, a resulting set $\{1,5\}$ cannot be explained by any pre-computed permutations and is reported. SNEK uses a Pseudo-Random Number Generator (PRNG) with a fresh seed to pick new operations. When it finds a violation, SNEK reports the seed used, which can later be used later to repeat the same test in isolation. SNEK this particular violation using 4 operations, automatically minimized to the 2 operations shown. To increase the odds of finding violations, SNEK launches many sets of N threads, which Figure 4 shows controlled with variable `WORKERS` on Line 37.

4.2.2 Non-deterministic return values. Some operations return non-deterministic values. For instance, Python's `set.__str__` method does not define an which order to print the elements in the set. Given the initial set in Line 10 of Figure 3, the strings `"{1, 2, 3, 4, 5, 6}"` and `"{6, 5, 4, 3, 2, 1}"` are both acceptable. To handle such cases, function `snek_op` takes an optional named argument `sorted` that calls Python's `sorted` on the result to handle non-determinism. In this case, both sets are sorted to the equivalent string `"{1, 2, 3, 4, 5, 6}"`.

It is still possible that some operations return a completely non-deterministic value that cannot be used to check for linearizability. For instance, method `set.__ior__`, equivalent to applying the operator `|=` to a set, returns the original set. As SNEK then further mutates the set in a non-deterministic way, the value that the set holds after such operation is non-deterministic. For such methods, SNEK provides the optional named argument `noreset` that simply discards the return value of the method. Using such methods is still useful if they perform destructive modifications to the underlying data structure, which `__ior__` does.

4.2.3 Minimizing tests. To maximize the odds of finding violations, SNEK runs many operations concurrently, one per thread (e.g., $N=4$ or more, instead of 2 as shown in Figure 4). As a result,

Table 5. Violations (Viols) found in 1h for all three *Data Structures (DS)* for all GIL-based Python implementations considered.

Data-structure	CPython	PyPy	GraalPy
Set	0	0	0
List	0	0	0
Dict	0	0	0

when SNEK finds a violation, there may be many unrelated operations in the execution. SNEK then attempts to eliminate all the operations that do not contribute to the violation observed using a straightforward approach: Pick one operation at random and remove it, rerun the remaining operations, and check if the violation is still present. If the execution is still *violation-revealing*, then the operation picked is clearly unrelated and can be eliminated. Otherwise, the execution is *non violation-revealing* and the operation cannot be eliminated. Once SNEK tries to eliminate all operations, it is left with the minimal number of operations required to observe the violation originally found.

However, such a straightforward approach suffers from false-positives and may fail to eliminate unrelated operations because the violation simply did not take place during a particular execution. To avoid such false-positives, SNEK's minimizer can be tuned with a parameter `AVOID_FP` as N that only keeps any operation O after observing a number N of non violation-revealing runs without O . Note that SNEK does not suffer from false-negatives. If SNEK removes a needed operation, the violation cannot be observed anymore, which causes SNEK's minimizer to always keep such a needed operation.

4.3 Using SNEK

This section describes how we used SNEK to find linearizability violations for leaf NoGIL operations (Section 4.1) on the same versions of the Python interpreters used in Section 3.

We wrote operations for SNEK (see Figure 3) to test lists, sets, and dictionaries. We started from the operations defined by each data-structure and decided if each operation is leaf NoGIL (Section 4.1), which resulted in all the operations in Tables 3 and 4. We wrote a `snek_op` for each such operation, with simple data (numbers and strings). Then, we built CPython with line coverage reporting, and focused on the C files that define each data structure. We added enough different operations to reach 50% coverage for dictionaries, and 80% coverage for lists and sets. The source file that holds the implementation of dictionaries also holds unrelated functionality (e.g., views over existing dictionaries and iterators), which limits the maximum coverage we can achieve. In the end, we wrote 97 operations for lists, 110 for sets, and 98 for dictionaries.

We tested all operations in our extensive set of non-GIL operations on all Python interpreters. We ran SNEK for 1h on x86 and AMD64 (same as Section 3) configured with 4 threads in parallel.

4.3.1 Linearizability of GIL-based Python interpreters. Table 5 summarizes our findings for all implementations based on a GIL. **SNEK did not find any violation of linearizability**, which provides strong empirical evidence that our formal model (Section 2.3) explains the atomicity provided by leaf NoGIL operations on existing GIL-based Python implementations.

4.3.2 Linearizability of Python interpreters without a GIL. Section 3 shows how current Python interpreters without a GIL still provide the same Sequential Consistency guarantee as GIL-based interpreters (except IronPython). Even though our model of Python execution does not target interpreters without a GIL, we repeated the linearizability checking to see if existing interpreters provide the same guarantees. Table 6 summarizes our findings.

Table 6. Violations (*Viols*) found in 1h for all three *Data Structures (DS)* for all non-GIL based Python implementations considered (CPython's experimental support for PEP 703, Jython, and IronPython), and number of unique minimizations requiring 2, 3, or 4 operations. The numbers in parentheses are how many original violations were minimized, so a cell 22 (1236) means that 1236 original violations were minimized to 22 unique violations.

DS	CPython PEP 703				IronPython				Jython			
	Viols	Minimized			Viols	Minimized			Viols	Minimized		
		2	3	4		2	3	4		2	3	4
Set	1306	22 (1236)	16 (31)	30 (39)	784	64 (766)	15 (15)	3 (3)	3269	296 (3232)	33 (36)	1 (1)
List	741	11 (607)	15 (110)	2 (24)	2720	83 (2646)	32 (72)	2 (2)	0	0	0	0
Dict	22	1 (22)	0	0	4463	64 (4356)	6 (10)	29 (33)	1917	94 (1927)	11 (12)	1 (1)

Interestingly, we note that Jython's lists are indeed linearizable. Jython implements lists using a coarse-grained approach with a single lock protecting all non-GIL list operations, which is similar to the implementation proposed in the text of PEP 703 (Section 4.6). Another observation is that SNEK's minimizer works well for all interpreters considered, reducing the vast majority of violations to only 2 operations. We believe that using a more aggressive configuration for SNEK can result in even more violations minimized (Section 4.5.2).

Given the recent development effort towards PEP 703 and the small number of violations, we analyzed all the violations in detail for CPython PEP 703 and reported them to the developers. Jython is in the process of being rewritten to support Python 3, so we limited our effort to a few violations only involving 2 threads which we also reported to the developers. IronPython is not Sequentially Consistent to start with (Section 3.1), and developers did not address our report. Given that we are not familiar with the code base, the large amount of violations found, and that we could not rely on the developers' knowledge to understand what went wrong, we did not analyze any IronPython violation. We describe the results below for CPython and Jython below.

Dictionary. We found a single violation with CPython dictionaries between operations `a.__len__`, which returns the number of key-pair values in `a`; and `a.__ior__(b)`, which adds all key-pairs from `b` to `a`, updating the value on `a` for keys also present in `b`, and returns `a`. When these two operations runs concurrently over the same dictionary `a` such that all keys in `b` are not in `a`, `__len__` can return values other than the length of `a` or the sum of the lengths `a+b`.

Jython implements dictionaries by mapping Python operations to equivalent Java operations on a linearizable `java.util.concurrent.ConcurrentHashMap`. All violations found for Jython dictionaries involve operations that do not map directly to an equivalent Java operation. As a result, Jython implements some Python operations as a non linearizable combination of many linearizable Java operations.

Lists. We found two violations with CPython lists. One violation involves `__len__`, which returns the number of elements in the list, and `sort`, which sorts the list destructively and returns `None`. In this case, `__len__` either returns the correct size or zero. The other violation involves `reverse`, which reverses the order of the elements in a list, and `__contains__`, which checks if an element is in a list. Attempting to reverse a list in thread `T1`, while another thread `T2` checks if the list contains a particular element can result `T2` not finding the element as `T1` is moving it.

Sets. We found several violations with CPython sets:

- We found the same violation involving `__length__` and other operations, as we describe above for dictionaries: `a.__ixor__(b)`, which changes `a` to hold elements present in either `a` or `b` and not both; `a.__symmetric_difference_update(c)`, which changes `a` to hold all

the elements that are in either `a` or `c` but not in their intersection; `a.update(d)`, which changes `a` to hold all elements present in either `a` or `d`; and `a.__isub__(e)`, which changes `a` to hold only the elements that are not also present in `e`. As with dictionaries, PEP 703 allows for implementations of length that result in this violation.

- Running method `a.__eq__(b)` concurrently with `a.__isub__(c)` on the same set `a` can return true when it should always return false: for a set `b` that holds different elements from `a`, and for a set `c` which difference from `a` is not the same as the contents of `b`. A particular example that SNEK found is `a={'a', 'b', 'c', 'd', 'e', 17, 18}`, `b={'a', 'b', 'c', 'd', 'e', 18}`, and `c={'b', 'd', 1, 17}`.
- We found a violation when method `a.intersection_update(b)` — which modifies `a` to hold the intersection with `b` — runs concurrently with either `a.symmetric_difference_update(b)` or `a.__isub__(c)` on the same set `a`.
- Finally, we found a violation when running `a.clear()`, which modifies `a` by removing all elements from it, concurrently with `a.__str__()`, which returns a string representation of `a`. In this case, it is possible for method `__str__` to return the string representation of an empty dictionary `"{}"` instead of the original set or an empty set `"set()"`.

Sets in Jython suffer from the same issue as dictionaries, described above.

4.4 Root cause analysis

In the process of reporting the issues found, our interactions with the developers shed some light on the underlying cause of some violations.

CPython clear and __str__ for sets. The CPython implementation of `__str__` first checks if the set is empty, returning `"set()"` in this case. Then, it uses an auxiliary function to create a list with all the elements in the set, which it then turns into a string of comma separated values, and finally surrounds with the characters `{` and `}`. However, the auxiliary function can unlock the set, and then quickly relock it. In the meantime, another thread calling `clear()` on the same set can remove all the elements, resulting in the incorrect string representation of `"{}"`. The developers fixed this issue by using a different auxiliary function to create the list of elements from the set, adapted the test included in our report and added it to CPython's test suite, and backported the issue to CPython 3.13.

CPython sort and __len__ for lists. When sorting a list in Python, developers can provide custom key argument to “*extract a comparison key from each list element*” [36]. Such functions can mutate the list being sorted, which CPython developers characterize as “*a core-dump factory*”.³ As such, CPython considers any access to a list being sorted as undefined and artificially sets the size of any list being sorted as zero, making it appear empty to any custom key function that attempts to manipulate or modify the list while it is being sorted. Unfortunately, removing the GIL now makes other threads able to see the list as artificially empty while it is being sorted, which violates linearizability.

While discussing this issue with the developers, we proposed a fast-path algorithm for getting the length of a list that avoids locking in the common case when a list is not being sorted. The developers expressed concern over complicating the logic of the length field, and proposed an alternative sketch for a fix that moves the complexity to the locking/unlocking logic while sorting a list. However, the developers are not enthusiastic about fixing this behavior and the issue is still open at the time of writing.

³<https://github.com/python/cpython/blob/09d6f5dc7824c74672add512619e978844ff8051/Objects/listobject.c#L2831-L2839>

Table 7. Number of original issues found when varying the number of threads WORKERS. Each cell contains: total violations found, unique minimized violations, and violations reported as issues.

WORKERS	Set	List	Dict
1	13 / 10 / 2	80 / 22 / 2	1 / 1 / 1
10	174 / 54 / 3	177 / 35 / 2	1 / 1 / 1
100	74 / 30 / 3	163 / 31 / 2	3 / 2 / 1
1000	58 / 24 / 3	72 / 15 / 2	3 / 1 / 1

Table 8. Time and results of setting AVOID_FP to 1, 10, 100, and 200. The results show the average time of 10 runs, and the worst results observed for the minimizer.

		1	10	100	200
Set	Time	0.68 ± 0.19	2.39 ± 1.30	14.70 ± 10.96	27.39 ± 21.32
	2	397	1055	1236	1243
	3	410	183	31	24
	4	499	68	39	39
List	Time	0.78 ± 0.25	3.54 ± 2.00	19.82 ± 18.21	34.74 ± 33.65
	2	137	407	607	646
	3	317	227	110	83
	4	287	107	24	12
Dict	Time	0.73 ± 0.17	2.54 ± 1.27	12.40 ± 6.39	24.43 ± 13.25
	2	1	18	22	22
	3	8	3	0	0
	4	13	1	0	0

Jython difference and difference_update for sets. For most data structures, Jython maps each Python operation into an equivalent operation on a `java.util.concurrent` data structure, which is linearizable and thread-safe. However, there is no direct relationship between set differences and the version of Java used for Jython. As such, the Jython implementation used several set updates, which are not atomic between themselves. After reporting this issue, the developers agree that Jython should map each Python operation to one equivalent and atomic Java operation, and take special care for when this is not possible to ensure Jython behaves as CPython.

4.5 Tuning SNEK

SNEK's uses configuration parameters AVOID_FP and WORKERS as described above. In this section we evaluate their impact. We use the experimental CPython support for PEP 703 in version 3.14, we evaluate SNEK on lists, sets, and dictionaries, and we set the defaults AVOID_FP=100 and WORKERS=100. For brevity, we show only the results for x86_64, as the ARM results show the same trends.

4.5.1 Impact of WORKERS. To understand the impact of worker threads, used to amplify SNEK's ability to detect violations, we ran SNEK for 10 minutes using a different value for WORKERS: 1, 10, 100, 1000. Table 7 shows the results. We can see that 10 workers find the most violations for both sets and lists, and 1 worker misses one of the bugs we submitted. Adding more workers over 10 results in finding less violations. This happens because more workers take longer to finish, so SNEK is not able to explore as many combinations of operations.

4.5.2 Impact of AVOID_FP. SNEK's minimizer suffers from false-positives, which can be minimized via the parameter AVOID_FP as described in Section 4.2.3. We experimented different values (1, 10, 100, and 200) with the data we obtained from our 1h run, described in Section 4.3. Table 8 shows the results. We can see a dramatic increase in the quality of the minimized results with an increase in AVOID_FP up to 100 as the number of violations minimized to only 2 threads increases for all data structures. Increasing to 200 does not improve the results significantly.

4.6 Discussion

The results we present in this section show that our model for GIL-based Python interpreters aligns with the implementation of such interpreters, as we did not find any violations of linearizability. Of course, similarly to using litmus testing to test the memory model (Section 3) testing alone cannot prove that the implementations are indeed linearizable. However, we consider that the results

we obtained after such extensive testing are a strong indication that the underlying GIL-based interpreters match the model we propose.

All interpreters without a GIL showed some violations of linearizability. Such behaviors are not possible on GIL-based implementations (including the reference implementations CPython), and may introduce new unexpected behaviors on existing concurrent Python programs that assume atomicity of lists, sets, and dicts. We show in Section 5 that existing Python code indeed relies on such atomicity.

The documentation of IronPython [11] and Jython [21] do not describe the safety of sharing built-in data structures between threads. PEP 703 [15] discusses builtin Python data structures as part of “*Container Thread-Safety*”, in particular: lists, sets, and dictionaries. The PEP describes situations in which the GIL does not guarantee operation atomicity for programs that define custom iterators or comparators. The PEP then proposes using “*per-object locks to provide many of the same protections that the GIL provides*”. To that extent, it splits operations in three classes:

- (1) **All** operations that **modify** the data-structure must hold the respective object’s lock
- (2) **Most** operations that **read** a data-structure must hold the respective object’s lock
- (3) **A few** operations that **read** a data-structure can “*optimistically avoid locking to improve performance*”: accessing an index of a list, looking up a key on a dictionary, checking if a list contains an item, and using list iterators.
- (4) The length operation is the sole **exception** on all three data-structures that can be implemented without holding the lock. The PEP explains that it “*can be implemented directly with atomic accesses and do not need locks because they only access a single field*”.

PEP 703 matches the same atomic semantics of GIL-based interpreters (Section 2.3) to a great extent, but not completely. Our definition of non-leaf NoGIL operations, which are not atomic (Corollary 2.24), matches the PEP’s lack of atomicity for custom iterators or comparators. Our definition of leaf NoGIL functions matches the semantics of per-object locking defined in Classes 1–3 above, assuming the optimistic locking in Class 3 still ensures linearizability.

There are two important exceptions in which our model and the PEP disagree. *First*, we consider operation length as a leaf NoGIL operation, but the PEP allows it to observe intermediate results of concurrent operations by reading a field directly using atomic memory operations instead of locks. *Second*, operations on two objects that only acquire the lock on one object can cause concurrent operations to observe intermediate changes. The PEP explains that the per-object lock mechanism provides “*weaker protections than the GIL*” with “*mutual exclusion limited to individual objects*”. Our interpretation is that the per-object lock protects only the object being manipulated. For instance, consider that Thread T_1 calls `set1.__isub__(set2)` (equivalent to `set1 -= set2`, modifies `set1` to hold the difference of `set1` and `set2`) and Thread T_2 calls `set2.clear()` concurrently. With the GIL, T_1 can either see `set2` with the original elements or empty. Without the GIL, both threads acquire different locks, so T_1 can see `set2` with less elements than originally but not empty.

5 Uses of Concurrent Python in the wild

In this document, we propose a formal model which shows that the memory model of GIL-based Python interpreters if Sequential Consistency (SC), in Section 2.2, and that explains the conditions under which native/built-in function calls are atomic, in Section 2.3. This section shows that our proposed model aligns with the expectations of developers of concurrent Python code.

To find uses of concurrent Python, we searched Github for repositories that contain Python code and reference the threading module, then sorted by popularity (numbers of stars), and then we inspected the top 100 repositories for how threads manipulate shared state. Table 9 shows the results.

Table 9. Uses of the threading module found on Github, on the top 100 repositories with the most stars.

Not applicable	Repos	Synchronized	Repos	Unsynchronized	Repos
No threads	14	Locks	12	Shared variable	19
External threads	4	Queues	14	List	8
No shared state	24			Set	1
				Dictionary	4
TOTAL	42		26		32

Our results show that 42 repositories are **not applicable** to our study: 4 use threads from an external library (e.g., QT or boost, via native code), 14 do not use actual threads in their source, and 24 do not share any state between threads. A common pattern are programs that use multiple threads to download the contents of a particular website. Even though Python only runs one thread at a time (i.e., all executions are GIL-well-formed as per Definition 2.7), using multiple threads can multiplex I/O (in *blocking* functions as per Definition 2.19), resulting in many parallel network requests that lower the time needed to download the intended contents. In this case, such programs typically use a thread pool and start one thread per URL, which then downloads the contents and saves them to disk without interacting with any other thread.

Our results show that 26 repositories use **synchronization** to protect the state shared between threads, either with locks (12) or using a thread-safe multi-producer multi-consumer queue, introduced in Python 3 [37], that allows threads to exchange information safely (14).

Finally, our results show that 32 repositories **do not use any synchronization** to protect the state shared between threads. A common pattern is starting multiple threads that execute until a flag is set by another thread. Such use of shared variables (19) requires Sequential Consistency as the underlying memory model, otherwise the worker threads may never see the flag change values. We also found downloaders, as explained above, and generic servers that keep important data about the currently active connections, or the remaining URLs to download, in lists (8), sets (1), and dictionaries (4) shared among many worker/client threads. Such programs expect their use of *leafNoGIL* operations (Definition 2.19) to be atomic between threads (Theorem 2.23).

These results provide strong empirical evidence that developers find value in writing Python code that uses multiple threads, even if the most popular Python implementations still rely on a GIL that severely limits the applicability of threads as compared to other languages. Furthermore, these results show that existing multi-threaded code expects Python to behave as we describe in Section 2.1, matching the semantics of GIL-based implementations. Our results suggest that changing the GIL-based Python semantics can break existing concurrent Python code. On the other hand, the GIL-based semantics we propose guarantee that all Python implementations behave as expected by developers, regardless of whether each Python implementation uses a GIL to control concurrency.

6 Related Work

Testing memory models via the results of repetitive executions of short programs, consisting of sequences of reads and writes of shared variables, is well known. Initially, such programs were written by hand [9, 17], then formalized [5], then generated automatically from templates [4] and from specifications of memory models [45]. Besides focusing on CPU memory models, recent work uses litmus tests on GPUs [3, 23] and programming languages such as C/C++ [13, 28, 46], Java [31], and Kotlin [26].

Line-Up [8] is a model checker that most resembles SNEK. It checks for linearizability by enumerating all possible sequential behaviors of a set of method calls, and then checks for violations

of linearizability if the result of a possible thread interleaving was not enumerated. Line-Up is implemented on top of CHESS [29], another model checker for C# programs that can enumerate all possible thread schedulings and exhaustively run the same test on thread schedulings guaranteed to be different. SNEK, on the other hand, requires no control over the thread scheduler, relying on its fast execution speed to explore different possible schedulings on a large number of threads executing concurrently. As a result, SNEK can find violations of linearizability in Python implementations written in C, Java, and C#.

Another tool [34] explores violations of linearizability for Java without enumerating all the sequential interleavings, instead creating tests by picking prefixes and suffixes, reusing prefixes, and combining them with new suffixes. SNEK simply picks operations at random and computes all the possible interleavings. Furthermore, [34] only detects violations that result in an exception being thrown, missing operations that violate linearizability by returning a wrong value or leaving the data-structure in an inconsistent state. As such, [34] cannot be used to detect violations of linearizability in Python implementations as SNEK does. Even though SNEK does not detect deadlocks, it considers exceptions as part of the data-structure behavior, thus also detecting any exception only thrown during a concurrent execution as a violation of linearizability.

SNEK is very similar to Violat [10], which in turn is similar to Line-Up. Violat considers schemas as sequential combinations of many operations in the same thread. Using the same terms, SNEK uses a schema of size 1. Violat then further refines the observed violations using off-the-shelf analysis for Java: JPF [44] and jctest [31]. Unfortunately, to the best of our knowledge, no such tools exist for Python. Violat is sound and complete as explained in the paper: “*the approach is both sound and complete, i.e., in the limit: for every consistency violation there is a failed execution of some test program, and every failed test witnesses an actual consistency violation*”. We believe SNEK offers the same soundness guarantees: every failed test is a witness of an actual consistency violation. Given that SNEK does not rely on analysis techniques, instead using repeated executions of the same test directly on the hardware, SNEK cannot offer the same completeness guarantee as it may miss consistency violations.

SNEK uses automatic delta-debugging to minimize programs, which is the starting approach of [39]. Given the complexity of the domain of [39], delta-debugging alone does not work as it may get stuck in local minima, which the authors address via domain-specific techniques to further minimize programs. We find that simple automatic delta-debugging works well for SNEK’s domain. Our test reduction technique is applied to a novel domain (minimizing violations of linearizability in multi-threaded Python), but uses well established techniques that are not novel themselves (automatic delta-debugging).

CAVE [43] finds the linearization points of library operations with a pure specification, and builds a proof of linearizability. The authors point out that most linearization points in existing algorithms do not change the underlying state (e.g., popping from an empty stack), and only one linearization point actually changes the state. As such, CAVE can prove the linearizability of such data structures by defining the abstract state of the data structure and defining abstract operations as pure if they do not modify the abstract state. SNEK tests data structures that keep internal state and use non-pure operations, but does not prove linearizability. As such, SNEK can be applied in contexts where CAVE is not applicable, in particular the implementation of built-in Python sets, lists, and dictionaries in C, Java, and C#.

7 Conclusion

In this paper, we describe Python’s *de-facto* GIL-based behavior as a memory model that observes *Sequentially Consistency* (SC), based on a formal model of a GIL-based language that matches Python’s semantics. Our model also explains the conditions under which native function calls are

atomic, which include most operations on lists, sets, and dictionaries. We validate that our model matches the observed behavior of existing GIL-based implementations using empirical data from litmus testing (memory model) and linearizability testing (builtin data-structures) on CPython, PyPy, and GraalPy. Furthermore, we manually analyze 100 popular Python repositories that use multiple threads, and find that developers indeed expect Python to be SC and builtin data-structures to be atomic as our model describes.

We extend our testing to Python implementations not based on a GIL: CPython's prototype for PEP 703 (which paves the way for removing the GIL from CPython), Jython, and IronPython. We find that only IronPython is not SC, exposing Python programs to the underlying .NET memory model. We also find that all builtin data-structures allow new behaviors impossible to observe while using a GIL. We reported our findings to the developers, and used the feedback to perform root-cause analysis on representative violations. One of our reports was a real bug that the developers fixed and backported to earlier Python version. All other issues remain open at the time of writing, as developers consider the implications of such behaviors.

8 Data Availability Statement

Upon acceptance, we will submit an artifact to the AEC that can replicate all the experiments (and respective results) described in this document. The artifact will contain:

- The C++ litmus tests, respective translation to Python 3.x and 2.x, and all the experimental scripts and source code of all Python implementations to replicate the results reported in Section 3.
- The prototype implementation of SNEK, together with our configurations used to perform the linearizability checking described of each Python data-structure described in Section 4, as well as all the experimental scripts to replicate all the results reported.
- A detailed list of all the Github issues reported in Section 4 (currently omitted to comply with the double-blind review process).
- A detailed list of all the Github repos manually analyzed, and the respective results for each repo, as reported in Section 5.

We are currently preparing a self-contained artifact (e.g., VM image with all the dependencies ready to execute), that we will submit to the AEC.

References

- [1] 2023. Python Official Discussion Forums: A fast, free threading Python. <https://discuss.python.org/t/a-fast-free-threading-python/27903/22?page=2>. Accessed: 2026-03-07.
- [2] Sarita V. Adve and Kourosh Gharachorloo. 1996. Shared Memory Consistency Models: A Tutorial. *Computer* (1996).
- [3] Jade Alglave, Mark Batty, Alastair F. Donaldson, Ganesh Gopalakrishnan, Jeroen Ketema, Daniel Poetzl, Tyler Sorensen, and John Wickerson. 2015. GPU Concurrency: Weak Behaviours and Programming Assumptions. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems* (Istanbul, Turkey) (ASPLOS '15). Association for Computing Machinery, New York, NY, USA, 577–591. doi:10.1145/2694344.2694391
- [4] Jade Alglave, Luc Maranget, Susmit Sarkar, and Peter Sewell. 2010. Fences in weak memory models. In *Proceedings of the 22nd International Conference on Computer Aided Verification* (Edinburgh, UK) (CAV'10). Springer-Verlag, Berlin, Heidelberg, 258–272. doi:10.1007/978-3-642-14295-6_25
- [5] Jade Alglave, Luc Maranget, Susmit Sarkar, and Peter Sewell. 2011. Litmus: running tests against hardware. In *Proceedings of the 17th International Conference on Tools and Algorithms for the Construction and Analysis of Systems: Part of the Joint European Conferences on Theory and Practice of Software* (Saarbrücken, Germany) (TACAS'11/ETAPS'11). Springer-Verlag, Berlin, Heidelberg, 41–44.
- [6] Arm Ltd. [n.d.]. Arm Architecture Reference Manual for A-profile architecture. <https://developer.arm.com/documentation/ddi0487/latest>. Accessed: 2025-03-07.

- [7] William J. Bolosky and Michael L. Scott. 1993. False sharing and its effect on shared memory performance. In *USENIX Systems on USENIX Experiences with Distributed and Multiprocessor Systems - Volume 4* (San Diego, California) (Sedms '93). USENIX Association, USA, 3.
- [8] Sebastian Burckhardt, Chris Dern, Madanlal Musuvathi, and Roy Tan. 2010. Line-up: a complete and automatic linearizability checker. *SIGPLAN Not.* 45, 6 (June 2010), 330–340. doi:10.1145/1809028.1806634
- [9] William Collier. 1994. ARCHTEST. <http://www.mpdia.com/archtest.html>. Accessed: 2025-03-07.
- [10] Michael Emmi and Constantin Enea. 2019. Violat: Generating Tests of Observational Refinement for Concurrent Objects. In *Computer Aided Verification*, Isil Dillig and Serdar Tasiran (Eds.). Springer International Publishing, Cham, 534–546.
- [11] Michael Foord and Christian Muirhead. 2009. *IronPython in Action*. Manning Publications Co., USA.
- [12] Python Software Foundation. [n. d.]. CPython. <https://github.com/python/cpython>
- [13] Luke Geeson and Lee Smith. 2024. Compiler Testing with Relaxed Memory Models. In *Proceedings of the 2024 IEEE/ACM International Symposium on Code Generation and Optimization* (Edinburgh, United Kingdom) (CGO '24). IEEE Press, 334–348. doi:10.1109/CGO57630.2024.10444836
- [14] Google Inc. 2022. The Go Memory Model. <https://go.dev/ref/mem>. Accessed: 2025-03-07.
- [15] Sam Gross. 2023. PEP 703 – Making the Global Interpreter Lock Optional in CPython. <https://peps.python.org/pep-0703/>
- [16] Guido van Rossum. 2009. A Brief Timeline of Python. <https://python-history.blogspot.com/2009/01/brief-timeline-of-python.html>. Accessed: 2025-03-07.
- [17] Sudheendra Hangal, Durgam Vahia, Chaiyasit Manovit, and Juin-Yeu Joseph Lu. 2004. TSOtool: A Program for Verifying Memory Systems Using the Memory Consistency Model. In *Proceedings of the 31st Annual International Symposium on Computer Architecture* (München, Germany) (ISCA '04). IEEE Computer Society, USA, 114.
- [18] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: a correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (July 1990), 463–492. doi:10.1145/78969.78972
- [19] Thuan Quang Huynh and Abhik Roychoudhury. 2006. A memory model sensitive checker for C#. In *Proceedings of the 14th International Conference on Formal Methods* (Hamilton, Canada) (FM'06). Springer-Verlag, Berlin, Heidelberg, 476–491. doi:10.1007/11813040_32
- [20] ISO. 2011. *ISO/IEC 14882:2011 Information technology — Programming languages — C++*. International Organization for Standardization, Geneva, Switzerland. 1338 (est.) pages.
- [21] Josh Juneau, Jim Baker, Frank Wierzbicki, Leo Soto, and Victor Ng. 2010. *The Definitive Guide to Jython: Python for the Java Platform* (1st ed.). Apress, USA.
- [22] Leslie Lamport. 1979. How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs. *IEEE Trans. Comput.* C-28, 9 (1979), 690–691. doi:10.1109/TC.1979.1675439
- [23] Reese Levine, Mingun Cho, Devon McKee, Andrew Quinn, and Tyler Sorensen. 2023. GPUHarbor: Testing GPU Memory Consistency at Large (Experience Paper). In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) (ISSTA 2023). Association for Computing Machinery, New York, NY, USA, 779–791. doi:10.1145/3597926.3598095
- [24] Lun Liu, Todd Millstein, and Madanlal Musuvathi. 2017. A volatile-by-default JVM for server applications. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 49 (Oct. 2017), 25 pages. doi:10.1145/3133873
- [25] Lun Liu, Todd Millstein, and Madanlal Musuvathi. 2019. Accelerating sequential consistency for Java with speculative compilation. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (PLDI 2019). Association for Computing Machinery, New York, NY, USA, 16–30. doi:10.1145/3314221.3314611
- [26] Denis Lochmelis, Evgenii Moiseenko, Yaroslav Golubev, and Anton Podkopaev. 2025. LitmusKt: Concurrency Stress Testing for Kotlin. arXiv:2501.07472 [cs.PL] <https://arxiv.org/abs/2501.07472>
- [27] Jeremy Manson, William Pugh, and Sarita V. Adve. 2005. The Java memory model. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Long Beach, California, USA) (POPL '05). Association for Computing Machinery, New York, NY, USA, 378–391. doi:10.1145/1040305.1040336
- [28] Robin Morisset, Pankaj Pawan, and Francesco Zappa Nardelli. 2013. Compiler testing via a theory of sound optimisations in the C11/C++11 memory model. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Seattle, Washington, USA) (PLDI '13). Association for Computing Machinery, New York, NY, USA, 187–196. doi:10.1145/2491956.2491967
- [29] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtii. 2008. Finding and reproducing Heisenbugs in concurrent programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (San Diego, California) (OSDI'08). USENIX Association, USA, 267–280.
- [30] .NET Foundation. [n. d.]. IronPython, the Python programming language for .NET. <https://ironpython.net/>. Accessed: 2025-03-07.

- [31] OpenJDK. [n. d.]. Java Concurrency Stress (jctstress). <https://github.com/openjdk/jctstress>. Accessed: 2025-03-07.
- [32] Oracle. [n. d.]. The Java Language Specification, Chapter 17: Threads and Locks. Online; accessed 2026-02-24.
- [33] Oracle Corp. 2025. GraalPy: A high-performance embeddable Python 3 runtime for Java. <https://www.graalvm.org/python/>. Accessed: 2025-03-07.
- [34] Michael Pradel and Thomas R. Gross. 2012. Fully automatic and precise detection of thread safety violations. *SIGPLAN Not.* 47, 6 (June 2012), 521–530. doi:10.1145/2345156.2254126
- [35] Python Software Foundation. [n. d.]. Jython: Python for the Java Platform. <https://ironpython.net/>. Accessed: 2025-03-07.
- [36] Python Software Foundation. 2025. Python 3.13.2 Documentation - The Python Standard Library - Built-in Types. <https://docs.python.org/3/library/stdtypes.html#sort>. Accessed: 2025-03-07.
- [37] Python Software Foundation. 2025. queue — A synchronized queue class. <https://docs.python.org/3/library/queue.html>. Accessed: 2025-03-07.
- [38] Python Steering Committee. [n. d.]. PEP 703 (Making the Global Interpreter Lock Optional in CPython) acceptance. <https://discuss.python.org/t/pep-703-making-the-global-interpreter-lock-optional-in-cpython-acceptance/37075>. Accessed: 2025-03-07.
- [39] John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation* (Beijing, China) (PLDI '12). Association for Computing Machinery, New York, NY, USA, 335–346. doi:10.1145/2254064.2254104
- [40] Armin Rigo and Samuele Pedroni. 2006. PyPy's approach to virtual machine construction. In *Companion to the 21st ACM SIGPLAN Symposium on Object-Oriented Programming Systems, Languages, and Applications* (Portland, Oregon, USA) (OOPSLA '06). Association for Computing Machinery, New York, NY, USA, 944–953. doi:10.1145/1176617.1176753
- [41] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O. Myreen. 2010. x86-TSO: a rigorous and usable programmer's model for x86 multiprocessors. *Commun. ACM* 53, 7 (July 2010), 89–97. doi:10.1145/1785414.1785443
- [42] Robert F. Stärk and Egon Börger. 2004. An ASM Specification of C# Threads and the .NET Memory Model. In *Abstract State Machines 2004. Advances in Theory and Practice*, Wolf Zimmermann and Bernhard Thalheim (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 38–60.
- [43] Viktor Vafeiadis. 2010. Automatically proving linearizability. In *Proceedings of the 22nd International Conference on Computer Aided Verification* (Edinburgh, UK) (CAV'10). Springer-Verlag, Berlin, Heidelberg, 450–464. doi:10.1007/978-3-642-14295-6_40
- [44] Willem Visser, Corina S. Pundefinedsundefinedreanu, and Sarfraz Khurshid. 2004. Test input generation with java PathFinder. In *Proceedings of the 2004 ACM SIGSOFT International Symposium on Software Testing and Analysis* (Boston, Massachusetts, USA) (ISSTA '04). Association for Computing Machinery, New York, NY, USA, 97–107. doi:10.1145/1007512.1007526
- [45] John Wickerson, Mark Batty, Tyler Sorensen, and George A. Constantinides. 2017. Automatically comparing memory consistency models. *SIGPLAN Not.* 52, 1 (Jan. 2017), 190–204. doi:10.1145/3093333.3009838
- [46] Matt Windsor, Alastair F. Donaldson, and John Wickerson. 2022. High-coverage metamorphic testing of concurrency support in C compilers. *Software Testing, Verification and Reliability* 32, 4 (2022), e1812. arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/stvr.1812> doi:10.1002/stvr.1812
- [47] Thomas Würthinger, Christian Wimmer, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Christian Humer, Gregor Richards, Doug Simon, and Mario Wolczko. 2013. One VM to rule them all. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software* (Indianapolis, Indiana, USA) (Onward! 2013). Association for Computing Machinery, New York, NY, USA, 187–204. doi:10.1145/2509578.2509581

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009